

Due date: **Sep. 22, 2026, 11:59 PM** (Indy time). The penalty for late submission will be applied goes as follows for each subsequent 24 hour period: {10%; 25%; 50%}. That is, if you submit this assignment on a Monday at 4:00 AM in the morning (if it were due on Friday at 11:59 PM), your submission is three days late.

What to turn in:

1. Your submission should include your complete code base in an archive file (zip, tar.gz) and q1/, q2/, q3/, q4/, q5/, and a **very very clear README describing how to run it**.
2. **A brief report (typed up in LaTeX, submit as a PDF file, NO handwritten scanned copies) describing what you solved, implemented and known failure cases if any. Report will be the first thing that we will look at from your submission.**
3. Submit your entire code and report to Canvas

Notes from instructor:

- Start early!
- You may ask the TA or instructor for suggestions and discuss the problem with others (minimally). But **all parts of the submitted code must be your own**.
- Use MATLAB or Python for your implementation. Do not use pre-defined functions. For Python users, you may use libraries minimally to load data or perform basic operations such as matrix multiplications.
- Make sure that the TA can easily run the code by plugging in our test data.

## Problem 1

(Sampling, **10pts**) Before we do any experiment in this homework, make sure that you use gray scale normalized image for your experiment, i.e., your image intensity should range  $[0, 1]$ . You may use built-in functions for image normalization. Implement your own function to downsample an image into half size (for x and y directions) by sampling every other pixel. Perform downsampling twice on 'lena.png'. Compare the downsampled images with the original in the same actual size (pixel size for low resolution images should be bigger.) How do they look? Now, perform upsampling of the image by inserting an empty pixel between every and each pixel. Run it twice to get it back to the original size. How are the upsampled images looking? Write your observation in the report.

## Problem 2

(Gaussian smoothing, **30pts**) Implement a function that takes in an image  $I$ , kernel size  $k$  and scaling parameter  $s$  (i.e.,  $\sigma$ ) and outputs its smoothed version after Gaussian smoothing, i.e.,  $I_{\text{smooth}} = \text{myGaussianSmoothing}(I, k, s)$ . Test it on the given image 'lena.png'. Change the

kernel size to  $k = \{3; 5; 7; 11; 51\}$  with fixed  $s = 1$ . What changes in the result do you see? Now, change the  $s = \{0.1; 1; 2; 3; 5\}$  with fixed kernel size  $k = 11$ . How is the result changing? Explain what you did and a brief report on your observation.

## Problem 3

(Image filtering, **20pts**) Let's solve the problem that occurred from sampling. Everytime after performing upsampling, perform Gaussian smoothing with  $k = 11$  and  $\sigma = 1$ . Did it get better? How about we perform median filtering? Implement your own median filter and apply it on the same upsampled data. Did it improve the result? Which filtering is better? Discuss your observation in the report.

## Problem 4

(Noise, **20pts**) On the original 'lena.png' image, let's add some Gaussian noise  $\sim N(0; 0.1)$  at each pixel, i.e.,  $I_{\text{noisy}}(x; y) = I(x; y) + r$  where  $r \sim N(0; 0.1)$ . How is the image looking? Now, let's perform Gaussian smoothing with any parameter you like that works. Did the result improve? Median filter in the previous question probably made you frustrating, but let's give it one more chance. Apply median filtering and see how the result shows. Now, let's go change the type of noise you added. Before you add it to the original image, set the noise to value 1 everywhere where the noise was  $> 0.2$  and 0 otherwise. Apply Gaussian smoothing and see what you get. Apply median filtering as well and see what you get. Discuss your observation in the report.

## Problem 5

(Sobel filters, **20pts**) Implement a Sobel filter that calculates gradient along x and y directions. To visualize our sobel operator, normalize the gradients so that all the values lie between  $[0,1]$ . Your function should look like  $[mag, ori] = \text{mySobelFilter}(I)$  that takes an image  $I$  as an input and output edge response (magnitude) and orientation for each pixel. Try visualizing the result in color by using the magnitude to specify the saturation and value of an image and the orientation to specify the hue. We haven't talked HSV space in the class but can be found at [https://en.wikipedia.org/wiki/HSL\\_and\\_HSV](https://en.wikipedia.org/wiki/HSL_and_HSV). Discuss your observation in the report.